

RACE: A Rapid ARM Cryptographic Engine for Code-Based Classic McEliece PQC Scheme

Wen Wu, Jiankuo Dong*, Xuecheng Liu, Shuzhou Sun, Zhenjiang Dong, Jingqiang Lin, Fu Xiao

Abstract—With the rapid development of quantum computing, traditional public-key cryptosystems are increasingly vulnerable, making post-quantum cryptography (PQC) a critical area for securing future information systems. As a prominent code-based key encapsulation mechanism (KEM), Classic McEliece offers strong quantum security. However, its large public key size and complex decoding process introduce significant performance bottlenecks, hindering its practical deployment on mobile and edge devices. To address these challenges, we propose RACE (Rapid ARM Cryptographic Engine), a systematic acceleration framework tailored for ARMv8 architectures, designed to enhance the efficiency of Classic McEliece while preserving its constant-time security guarantees. In the key generation phase, we introduce an implicit data layout and in-place transformation strategy based on LUP decomposition, coupled with NEON SIMD vectorization. For the encapsulation phase, we optimize error vector generation with constant-time techniques and batch-loading strategies to reduce memory access redundancy during matrix-vector multiplications. In the decapsulation phase, we apply lane-level fusion and dual-lane butterfly fusion techniques, leveraging NEON instructions to parallelize field multiplication and fast fourier transform (FFT) butterfly operations. Experiments on three ARMv8 platforms (Kunpeng 920, Apple M1, Apple M2 Pro) demonstrate significant speedups of RACE compared to the official vec implementation. For the McEliece8192128 parameter set, key generation is accelerated by $1.33\times$, $1.90\times$, and $2.04\times$, encapsulation by $1.33\times$, $1.23\times$, and $1.26\times$, and decapsulation by $1.27\times$, $1.68\times$, and $1.66\times$, respectively. RACE also outperforms open-source libraries, particularly in decapsulation, where speedups reach several hundred times. These results validate the practical applicability and deployment potential of RACE in mobile and edge computing environments.

Index Terms—Post-quantum Cryptography, Key Encapsulation Mechanism, Classic McEliece, ARM, Cryptographic Engineering.

I. INTRODUCTION

The rapid advancement of quantum computing technologies has posed a profound threat to traditional public-key cryptosystems based on integer factorization and discrete logarithm problems, such as RSA and ECC [1]. The emergence of Shor's algorithm renders these classical schemes vulnerable in the presence of large-scale quantum computers, thereby motivating extensive research in the field of Post-Quantum Cryptography (PQC). PQC aims to design cryptographic algorithms that remain secure against quantum adversaries. Among various PQC candidates, code-based Key Encapsulation Mechanisms (KEMs) have drawn considerable attention due to their

strong theoretical foundations and long-standing resistance to cryptanalytic attacks. In particular, the Classic McEliece scheme [2], as a representative code-based KEM, stands out as one of the most promising candidates owing to its mature mathematical structure and decades of security evaluation.

The Classic McEliece algorithm is rooted in the original McEliece cryptosystem proposed in 1978, which is based on binary Goppa codes. Over the past several decades, it has demonstrated exceptional robustness against both classical and quantum attacks and has advanced to Round 4 of the NIST PQC standardization process [3]. Nevertheless, the main limitation of Classic McEliece lies in its computational intensity. Its key generation and decapsulation stages involve large-scale matrix operations, field arithmetic, and error-correcting code procedures, which lead to significant demands on memory and computational resources. Hence, improving the implementation efficiency of Classic McEliece without compromising its security is a critical step toward its practical deployment.

The ARMv8 architecture, widely adopted in mobile and edge computing platforms, offers a favorable balance between performance and energy efficiency. It features a flexible pipeline design and supports advanced SIMD instruction sets such as NEON, making it well-suited for Internet of Things (IoT) and mobile security applications. The 64-bit AArch64 extension further enhances computational throughput by providing a richer set of general-purpose registers and an improved instruction-level parallelism mechanism. Combined with its hierarchical memory structure and efficient register organization, ARMv8 presents substantial potential for parallelization, data rearrangement, and memory optimization, thereby offering a promising hardware platform for accelerating cryptographic algorithms such as Classic McEliece.

A. Related Work

With the growing demand for efficient PQC schemes, numerous studies have focused on optimizing PQC for ARM architectures, particularly ARMv8 processors. Sanal et al. [4] introduced optimized implementations of the Kyber on 64-bit ARM Cortex-A processors, improving the Number Theoretic Transform (NTT), noise sampling, and AES-based symmetric operations for significant performance gains. Ortiz et al. [5] implemented and benchmarked the Kyber and Saber on ARMv8 processors using NEON instructions to accelerate key generation, encapsulation, and decapsulation. Zhao et al. [6] optimized Kyber on ARMv8-A processors by leveraging NEON SIMD instructions to accelerate Barrett and Montgomery reductions as well as NTT/INTT operations. Aguilera

W. Wu, J. Dong, X. Liu, Z. Dong and F. Xiao are with School of Computer Science, Nanjing University of Posts and Telecommunications. S. Sun is with Huawei Technologies. J. Lin is with School of Cyber Security, University of Science and Technology of China.

*Corresponding author: Jiankuo Dong, djiankuo@njupt.edu.cn.

et al. [7] targeted the ML-DSA scheme, applying NEON-based acceleration to hash-related operations, which led to notable gains across key generation, signing, and verification. Kim et al. [8] optimized Falcon for ARMv8 processors by parallelizing FFT and NTT polynomial multiplications using the NEON. Kwon et al. [9] optimized FrodoKEM on ARMv8 by implementing parallel matrix multiplication and utilizing the built-in AES accelerator to improve computational efficiency.

However, existing works on Classic McEliece are still limited, particularly regarding performance tuning on 64-bit ARMv8 processors. Chen and Chou [10] introduced a constant-time Classic McEliece implementation for the ARM Cortex-M4 on the stm32f4-Discovery board, using flash memory for public key storage due to limited SRAM. Roth et al. [11] proposed an optimized implementation of Classic McEliece on ARM Cortex-M4 to address memory constraints by streaming and reconstructing the public key on demand. Sim et al. [12] presented an optimized implementation of the multiplication operations in the Classic McEliece on the ARMv8 processor, exploiting vectorization and instruction-level parallelism to significantly improve performance. In addition to ARM-based implementations, several studies have explored the optimization of the Classic McEliece scheme on other hardware platforms, including FPGAs [13]–[16] and GPUs [17], [18]. Such implementations exploit platform-specific features, such as hardware parallelism and optimized memory hierarchies, to mitigate the high computational cost inherent in code-based cryptography.

B. Contribution

We present a systematic optimization framework, RACE (Rapid ARM Cryptographic Engine), to address the performance bottlenecks of the Classic McEliece KEM on ARMv8 platforms. By leveraging algorithmic characteristics and architectural features, we significantly enhance the execution efficiency of Classic McEliece through vectorization, parallelization, and data layout optimizations. The contributions are summarized as follows:

- Firstly, we propose a series of optimization strategies targeting the computational bottlenecks of the Classic McEliece on ARMv8 platforms. In the key generation phase, we combine implicit data layouts from LUP decomposition with vectorization techniques, utilizing NEON SIMD instructions to substantially improve computational efficiency. In the encapsulation phase, we optimize error vector generation by employing batch loading strategies to reduce memory access redundancy in matrix-vector multiplications, thereby enhancing overall throughput. In the decapsulation phase, we achieve cross-channel parallel acceleration of finite field multiplication and FFT butterfly operations, resulting in significant optimization of core operator performance.
- Secondly, a comprehensive analysis of the computational characteristics of the Classic McEliece algorithm is conducted, and we propose a strategy that combines vectorization and batching across operation levels and channels. This includes techniques such as masked row

swapping, batched finite field multiplications, and dual-channel butterfly fusion. This approach fully exploits the parallelism and register bandwidth of the ARMv8 NEON SIMD instruction set, improving instruction-level parallelism, optimizing cache locality, and maintaining strict constant-time execution characteristics, thus significantly enhancing algorithmic efficiency while ensuring security.

- Finally, we conduct extensive performance evaluations on three ARMv8 platforms (Kunpeng 920, Apple M1, and Apple M2 Pro) to validate the effectiveness and scalability of the proposed optimizations across key generation, encapsulation, and decapsulation phases. Experimental results demonstrate that, while maintaining the correctness and security of the algorithm, the optimized implementation outperforms both the official vectorized implementation and existing open-source libraries (e.g., liboqs and PQClean), offering significant performance improvements and enhancing the practical deployment value of Classic McEliece on ARM-based platforms.

The remaining sections are as follows. Section II introduces the preliminaries. Section III details the strategies for optimizing Classic McEliece on ARMv8 platforms. Section IV presents the experimental evaluation and provides a comparative analysis with related work. Section V concludes the paper.

II. PRELIMINARIES

This section introduces the theoretical foundations of the Classic McEliece cryptosystem and the relevant architectural features of the ARMv8 platform, which together provide the basis for performance optimization.

A. Classic McEliece Parameters

Classic McEliece is based on the Niederreiter variant of the original McEliece cryptosystem, whose security relies on the computational hardness of the Random Linear Code Decoding (RLCD) problem. It employs a construction based on binary Goppa codes. Each private key corresponds to a binary Goppa code of length n , denoted as

$$\Gamma_2(g, \alpha_1, \dots, \alpha_n) = \left\{ c \in \mathbb{F}_2^n \mid \sum_i \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g} \right\},$$

where $g(x) \in \mathbb{F}_{2^m}[x]$ is a monic irreducible polynomial of degree t , and $\alpha_1, \dots, \alpha_n$ are distinct elements in $\mathbb{F}_{2^m}$ satisfying $g(\alpha_i) \neq 0$ for all i . The corresponding Goppa code has a dimension of $k = n - mt$ and is capable of correcting up to t bit errors. Table I summarizes the standard parameter sets of the Classic McEliece, including the parameters m , n , and t , as well as the corresponding sizes (in bytes) of the public keys, private keys, and ciphertexts.

The official Classic McEliece software provides four implementations for each parameter set: a reference version (`ref`) prioritizing code clarity over performance; a scalar-vectorized version (`vec`) using 64-bit integers; and two x86-specific versions employing Intel/AMD intrinsics for 128-bit `sse` and 256-bit `avx`, respectively. Notably, these implementations are exclusively optimized for x86 architectures, with no official ARM-specific variants provided.

TABLE I
CLASSIC McELIECE PARAMETER SETS

parameter set	m	n	t	public key	private key
McEliece348864	12	3,488	64	261,120	6,492
McEliece460896	13	4,608	96	524,160	13,608
McEliece6688128	13	6,688	128	1,044,992	13,932
McEliece6960119	13	6,960	119	1,047,319	13,948
McEliece8192128	13	8,192	128	1,357,824	14,120

B. Classic McEliece KEM

The Classic McEliece KEM can be abstracted into key generation, encapsulation, and decapsulation, as illustrated in Algorithm 1. In the key generation phase, given the security parameters m , n , and t , a binary Goppa code $\Gamma_2(g, \alpha_1, \dots, \alpha_n)$ of length n is constructed, and its systematic parity-check matrix H' is generated as the public key. The secret key comprises the polynomial $g(x)$, a permutation matrix P , and an invertible matrix S , which are used later to recover the original code structure during decapsulation. During encapsulation, a random error vector $e \in \mathbb{F}_2^n$ is used to compute the syndrome $s = H' \cdot e^T$ as the ciphertext, and the session key is derived as $K = \text{Hash}(e)$. In decapsulation, the receiver employs the secret key to decode the syndrome, recovering e , and derives the shared key $K' = \text{Hash}(e)$.

C. ARMv8 Architecture Overview

The ARMv8 architecture [19] is the latest mainstream 64-bit RISC design, widely deployed in mobile devices, edge computing, and IoT systems. Compared to earlier ARM architectures, ARMv8 offers significant improvements in register resources, instruction set extensions, and memory hierarchy. It introduces 64-bit general-purpose registers, enabling a larger address space and higher precision, while maintaining backward compatibility with 32-bit execution modes. The NEON SIMD extension provides 32 vector registers, each 128 bits wide, supporting parallel arithmetic and logical operations on multiple data elements, ideal for vectorized and matrix-based computations. ARMv8's multi-level cache structure, including L1 instruction and data caches and an L2 unified cache, reduces memory latency, while its out-of-order execution pipeline optimizes performance for compute- and memory-bound workloads. Additionally, ARMv8 includes efficient instruction-level optimizations such as bitwise logic operations and multiple-element load/store instructions, enabling low-overhead implementations of diverse computational patterns. This combination of efficient register file, cache subsystem, and advanced optimizations improves both performance and energy efficiency.

III. METHODOLOGY

As described in Section II-A, the official implementations of Classic McEliece include two versions, `ref` and `vec`, which do not rely on a specific hardware architecture. The `ref` implementation prioritizes code readability and conformity, aiming to provide a reference implementation of the algorithm rather than a performance-optimized version. In contrast,

Algorithm 1 Classic McEliece Key Encapsulation Mechanism

KEY GENERATION

Input: Security parameters (m, n, t)

Output: Public key pk , Secret key sk

- 1: Select a random polynomial $g(x) \in \mathbb{F}_{2^m}[x]$ of degree t
- 2: Generate $\alpha_1, \alpha_2, \dots, \alpha_n \in \mathbb{F}_{2^m}$ satisfying $g(\alpha_i) \neq 0$
- 3: Construct the parity-check matrix H of the Goppa code $\Gamma_2(g, \alpha_1, \dots, \alpha_n)$
- 4: Randomly generate an invertible binary matrix S and a permutation matrix P
- 5: Compute the intermediate matrix $\hat{H} = S \cdot H \cdot P$
- 6: Partition $\hat{H}$ as $\hat{H} = (M \mid \hat{T})$, where $M \in \mathbb{F}_2^{(n-k) \times (n-k)}$
- 7: **if** M is uninvertible **then**
- 8: Repeat from Step 4
- 9: **else**
- 10: Compute $H' = M^{-1}\hat{H} = (I_{n-k} \mid T)$
- 11: **return** $pk = T$, $sk = (S, P, g(x), \{\alpha_i\})$
- 12: **end if**

ENCAPSULATION

Input: Public key $pk = H'$;

Output: Ciphertext c , Shared key K ;

- 1: Randomly generate an error vector $e = (e_1, e_2, \dots, e_n) \in \mathbb{F}_2^n$ with Hamming weight t
- 2: Compute the syndrome $s = H' \cdot e^T$
- 3: Derive the session key $K = \text{Hash}(e)$
- 4: **return** Ciphertext $c = s$, Shared key K

DECAPSULATION

Input: Ciphertext $c = s$, Secret key $sk = (S, P, g(x), \{\alpha_i\})$;

Output: Recovered shared key K' ;

- 1: Compute the syndrome polynomial corresponding to s
- 2: Perform Goppa code decoding, using $g(x)$ and $\{\alpha_i\}$ to recover error vector e
- 3: **if** Decoding succeeds **then**
- 4: Compute $K' = \text{Hash}(e)$
- 5: **end if**
- 6: **return** K'

the `vec` implementation utilizes 64-bit integer vectorization, making it better suited for architectures such as ARMv8, which natively support 64-bit operations. Chen and Chou [10] initially ported the 64-bit `vec` implementation to a 32-bit version on an ARMv7 platform and subsequently conducted a series of performance optimizations. Given that the ARMv8 architecture natively supports 64-bit integer operations and features an efficient SIMD vector instruction set, we select the `vec` implementation as the baseline reference and further explore instruction-level parallelism and data access optimization strategies to enhance the overall computational efficiency of Classic McEliece on the ARMv8 platform.

A. Overall Structure

During the key generation phase, we address the efficiency bottleneck of Gaussian elimination by introducing a method that combines LUP decomposition, partial pivoting, and implicit storage strategies, effectively reducing both storage and computational overhead. Additionally, NEON vectorization is

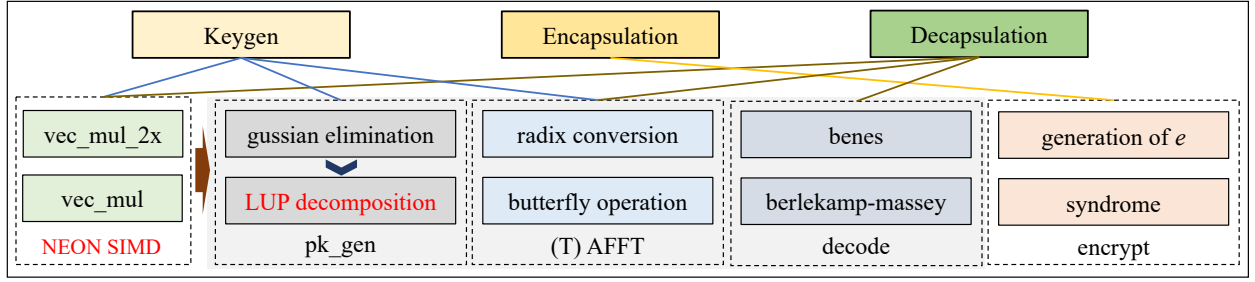


Fig. 1. Overall structure of the ARM platform-optimized Classic McEliece implementation.

applied to mask-controlled row swaps and row additions, further enhancing parallel processing capabilities. In the encapsulation phase, we optimize error vector generation and implement a batch loading strategy, improving matrix-vector multiplication by reducing redundant vector loads and enhancing memory access efficiency. The decapsulation phase, being computationally intensive, benefits from optimized finite field multiplication, using intra-multiplication vector parallelism (IMVP) and inter-multiplication batch parallelism (IMBP) mechanisms. NEON register channels are leveraged to merge multiple operations, significantly reducing instruction count and function call overhead. Moreover, in the additive FFT butterfly operations, we propose the dual-lane butterfly fusion (DLBF) strategy, restructuring the iteration to enable parallel execution of adjacent butterfly units, thus improving pipeline utilization and data locality. Figure 1 illustrates the overall structure of the ARM platform-optimized Classic McEliece implementation.

Furthermore, liboqs utilizes a performance benchmarking framework, `ds_benchmark.h`¹, where the timing mechanism on the ARMv8 platform employs a dual design that combines direct reading from hardware cycle counters with fallback to system-level time functions. This design ensures high-precision timing while maintaining portability and consistency across different hardware and operating systems, thereby ensuring the reproducibility and comparability of experimental results. Accordingly, we also adopt this framework.

B. Key Generation

In Classic McEliece, the core of key generation for the non-f parameter set involves transforming a randomly generated check matrix $\hat{H} = (M \mid \hat{T}) \in \mathbb{F}_2^{(n-k) \times n}$ into its systematic form $H' = (I_{n-k} \mid T)$, where the submatrix $M \in \mathbb{F}_2^{(n-k) \times (n-k)}$ is required to be invertible. Since the elements of matrix M are randomly generated, its invertibility is not guaranteed. Therefore, if M is found to be non-invertible, a new random matrix must be generated, and the process repeated. According to the official documentation, approximately 29% of the randomly generated matrices are successfully converted into systematic form.

In the public key generation, the Gaussian elimination algorithm is employed, with the primary objective of transforming

the left submatrix M into an identity matrix I_{n-k} over the finite field $\mathbb{F}_2$. During this process, an auxiliary operation matrix $O \in \mathbb{F}_2^{(n-k) \times (n-k)}$ is explicitly constructed and maintained to record all row transformation operations. Specifically, O is first initialized as the identity matrix, and during the Gaussian elimination and back-substitution procedures, all corresponding row operations applied to M are simultaneously performed on O , ensuring that $O \cdot M = I_{n-k}$ holds upon completion. Consequently, the systematic right submatrix T can be directly obtained through matrix multiplication as $T = O \cdot \hat{T}$, where $\hat{T}$ denotes the right submatrix prior to elimination. However, the main limitation of this approach lies in the explicit storage and maintenance of the operation matrix O , which introduces additional memory and computational overhead. Since O is of size $(n-k) \times (n-k)$, its storage cost amounts to approximately $(n-k)^2$ bits, leading to substantial memory consumption and increased memory access pressure for large parameter sets.

1) *LUP Decomposition for Key Generation*: We propose an optimization method for Gaussian elimination based on LUP decomposition, aimed at improving the efficiency of the elimination process and accelerating public key generation. If the matrix M is invertible, LUP decomposition computes a lower triangular matrix $L \in \mathbb{F}_2^{(n-k) \times (n-k)}$, an upper triangular matrix $U \in \mathbb{F}_2^{(n-k) \times (n-k)}$, and a permutation matrix $P \in \mathbb{F}_2^{(n-k) \times (n-k)}$, such that $P \cdot M = L \cdot U$. Compared to traditional Gaussian elimination, LUP decomposition offers significant storage advantages, as the matrices L and U are computed and stored within the storage space of matrix M , while the matrix P is stored as an index array of length $n-k$ (i.e., $r_0, r_1, \dots, r_{n-k-1}$). The LUP decomposition procedure we adopt is outlined in Algorithm 2.

In each round of elimination, we first scan the elements below the pivot in the current column and select the row with the first non-zero element as the pivot row, which is then swapped with the current row. Specifically, for each pivot position, if the pivot element is zero, the algorithm checks whether any non-zero elements exist below in the column. If such elements are found, the algorithm swaps the current row with the row containing the non-zero pivot. This “swap if necessary” strategy avoids redundant comparisons and swaps of candidate rows, reducing unnecessary memory accesses and branch evaluations, thereby improving the overall efficiency of the elimination stage.

To further reduce the computational and memory access costs associated with row exchanges, an index array `ind`

¹<https://gist.github.com/dstebila/6980008ec98209ef6075>

Algorithm 2 LUP Decomposition Used in [10].

Input: $H' \in \mathbb{F}^{(n-k) \times n}$. The first $n - k$ columns form the matrix $M \in \mathbb{R}^{(n-k) \times (n-k)}$

Output: $L^{-1}, U, P \in \mathbb{F}^{(n-k) \times (n-k)}$

```

1:  $B \leftarrow H'$ 
2: for  $\ell = 0$  to  $n - k - 1$  do
3:    $r_\ell \leftarrow \ell$ 
4: end for
5: for  $\ell = 0$  to  $n - k - 1$  do
6:   for  $i = \ell + 1$  to  $n - k - 1$  do
7:     if  $B_{i,\ell} = 1$  and  $B_{\ell,\ell} = 0$  then
8:       Swap rows  $i$  and  $\ell$  in  $B$ 
9:       Swap  $r_i$  and  $r_\ell$  in the row index array
10:    end if
11:  end for
12:  if  $B_{\ell,\ell} \neq 1$  then
13:    return  $\perp$  /* indicating failure */
14:  end if
15:  for  $i = \ell + 1$  to  $n - k - 1$  do
16:    for  $j = 0$  to  $w - 1$  do
17:      if  $j \neq \ell$  then
18:         $B_{i,j} \leftarrow B_{i,j} - B_{i,\ell} \cdot B_{\ell,j}$ 
19:      end if
20:    end for
21:  end for
22: end for
23:  $L^{-1} \leftarrow$  lower triangular of the first  $n - k$  columns of  $B$ 
24:  $U \leftarrow$  upper triangular of the first  $n - k$  columns of  $B$ 
25:  $P \leftarrow$  the row index array  $r_0, \dots, r_{n-k-1}$ 
26: return  $L^{-1}, U, P$ 

```

is introduced to record the row permutation relationships, eliminating the need for physical row exchanges within the matrix. In other words, matrix row swaps are replaced by updates to the `ind` array, which allows the logical order of matrix M to be represented by the permutation matrix P , such that $P \cdot M = L \cdot U$. This approach effectively avoids direct element swapping, significantly reducing memory access operations and improving data locality and overall computational performance.

During the “Eliminate Below” phase of Gaussian elimination, the matrix $P \cdot M$ is gradually transformed into an upper triangular form U , satisfying $U = L^{-1} \cdot (P \cdot M)$. Since operations are performed over the finite field $\mathbb{F}_2$, the non-diagonal elements of the lower triangular matrix L^{-1} , corresponding to the row elimination operations, can be stored implicitly in the lower triangular part of matrix M itself, without the need to explicitly construct or store L^{-1} . This implicit storage strategy not only saves additional memory space but also reduces the number of matrix accesses, enhancing the time and space efficiency of the entire elimination process.

After the LUP decomposition is complete, the systematic generation of the public key matrix T is achieved by sequentially applying the permutation matrix, the inverse lower triangular matrix, and the inverse upper triangular matrix. Specifically, the computation of the public key matrix T can

be expressed as $T = M^{-1} \cdot \hat{T} = (U^{-1} \cdot L^{-1} \cdot P) \cdot \hat{T}$, where $\hat{T}$ is the right submatrix prior to elimination. The entire process can be divided into three main steps. First, the permutation matrix P is applied to the right submatrix $\hat{T}$, resulting in $\hat{T}' = P \cdot \hat{T}$. Next, the inverse lower triangular matrix L^{-1} is applied to perform the corresponding row transformations, effectively utilizing the transformation information implicitly stored in the lower triangular part of matrix M from the elimination phase, yielding $\hat{T}'' = L^{-1} \cdot \hat{T}'$. Finally, the inverse upper triangular matrix U^{-1} is applied to perform back-substitution, producing the systematic public key matrix $T = U^{-1} \cdot \hat{T}''$. Through these steps, the public key matrix T is successfully generated.

These optimized approaches leverage the advantages of LUP decomposition to streamline the Gaussian elimination process, reduce memory usage, and improve overall computational efficiency in public key generation.

2) *Vectorized Optimization*: To further enhance the computational efficiency of the Gaussian elimination phase, we present a vectorized reorganization of core matrix operations based on the ARMv8 architecture, fully leveraging the parallel processing capabilities of the NEON instruction set. Operations such as row swapping, row addition, and column permutation are executed through data-level parallelism, resulting in efficient execution while effectively reducing memory access overhead and instruction cycle consumption, all while preserving the algorithmic equivalence.

Traditional row swap operations rely on conditional branching and element-wise swaps, leading to branch prediction overhead and discontinuous memory access. We implement a masked row swap approach, utilizing NEON vector registers for dual-block parallel swaps. By performing XOR operations (`veorq_u64`) and logical AND (`vandq_u64`) with a mask, we selectively swap the required elements, thereby eliminating branch operations and achieving data exchange at the register level, maintaining constant-time behavior. During the “Eliminate Below” phase of elimination, we perform masked XOR accumulation on the target row, reducing the number of loop iterations and memory access pressure. Utilizing NEON’s dual-block loading and parallel logical operations (`vandq_u64`, `veorq_u64`), this approach efficiently executes the XOR operation on 64-bit data blocks, avoiding explicit branch checks and improving pipeline utilization. To facilitate the efficient application of the permutation matrix P in LUP decomposition, we design an index-based column permutation function, which records row swap relationships via an index array, avoiding costly physical row swaps. This strategy enhances memory locality while maintaining logical correctness. Public-key matrix extraction is achieved by parallel loading of the right-hand matrix blocks via NEON, followed by compressed output as a bitstream. For non-byte-aligned data, scalar storage ensures proper alignment, adhering to the standard data layout. Additionally, multiplication in $\mathbb{F}_{2^m}$ with modular reduction (`vec_mul`) has been vectorized, with further details provided in Section III-D.

Table II presents the performance comparison of public key generation between the official `vec` implementation and our optimized implementation on the same ARMv8 platform. It can be observed that the proposed approach achieves

TABLE II
EXECUTION TIME FOR PUBLIC KEY GENERATION

parameter set	vec (us)	ours (us)	improvement (%)
McEliece348864	9777.596	6491.378	33.61%
McEliece460896	30875.959	16255.357	47.35%
McEliece6688128	93215.788	69840.349	25.08%
McEliece6960119	83192.514	44175.044	46.90%
McEliece8192128	94187.250	33260.484	64.69%

consistently significant acceleration across all parameter sets, demonstrating its effectiveness and scalability in enhancing the efficiency of public key generation.

C. Encapsulation

The core of the encapsulation process in Classic McEliece lies in the execution of the Niederreiter encryption operation. Specifically, a random error vector $e \in \mathbb{F}_2^n$, drawn from a uniform distribution, is first generated, and its corresponding syndrome $s = H'e$ is computed. As described in Section II-B, Classic McEliece employs a systematic parity-check matrix $H' \in \mathbb{F}_2^{(n-k) \times n}$, which can be expressed as $H' = (I_{n-k} | T)$, where I_{n-k} is the $(n-k)$ -dimensional identity matrix, and $T \in \mathbb{F}_2^{(n-k) \times k}$. Consequently, the error vector e can be decomposed as $e = (e^{(0)} | e^{(1)})$, where $e^{(0)} \in \mathbb{F}_2^{n-k}$ represents the first $n-k$ components of e , and $e^{(1)} \in \mathbb{F}_2^k$ represents its remaining k components. The syndrome calculation is then simplified to $s = H'e = e^{(0)} + Te^{(1)}$. In the following, we will detail the generation process of the error vector e and the specific optimization techniques for the matrix multiplication $Te^{(1)}$.

1) *Generation of Error Vector e* : The generation of the error vector e involves randomly selecting t positions within an n -bit vector and setting these positions to 1, while all remaining positions are set to 0. In the official `vec` implementation, this process is achieved by performing mask generation and conditional bit-setting operations for each candidate index. The mask generation and block selection for each index are realized through a sequence of bitwise XOR and subtraction operations:

$$\text{mask} = i \oplus (\text{ind}[j] \gg 6), \quad \text{mask} -= 1, \quad \text{mask} \gg= 63,$$

after which the mask is applied to control the bit-setting operation within each 64-bit block. While this method is logically correct, it requires a complete conditional evaluation for every index and every block, resulting in a total computational complexity of $O(n \cdot t)$. In practice, this leads to significant redundancy in mask generation and memory access, causing high register pressure, low data path utilization, and ultimately limited overall performance.

To address these inefficiencies while maintaining semantic equivalence, we optimized the error vector generation procedure by eliminating per-block conditional mask evaluations. Specifically, the block index and intra-block offset are computed directly as

$$\text{block} = \text{ind}[j] \gg 6, \quad \text{bit} = \text{ind}[j] \& 63,$$

TABLE III
EXECUTION TIME FOR ERROR VECTOR e GENERATION

parameter set	vec (us)	ours (us)	improvement (%)
McEliece348864	15.629	11.755	24.79%
McEliece460896	37.671	29.691	21.18%
McEliece6688128	59.602	44.311	25.66%
McEliece6960119	48.498	34.310	29.25%
McEliece8192128	42.130	23.681	43.79%

and the corresponding bit within the block is set through a constant-time bitwise OR operation:

$$e_{\text{int}}[\text{block}] |= (1 \ll \text{bit}).$$

This design removes the need for conditional mask construction and shifting for each block, significantly reducing loop depth and arithmetic instruction count. Furthermore, by pre-initializing all blocks to zero and employing a contiguous memory write pattern, the optimization enhances both memory access locality and register utilization. Overall, the proposed approach reduces the per-index processing overhead from being proportional to the number of blocks to a constant-time complexity of $O(t)$, thereby effectively lowering memory latency and the total number of bitwise operations. As shown in Table III, this optimization achieves a substantial performance improvement in the error vector generation phase.

In addition, the `vec` implementation employs a sorting network to order the randomly generated indices, enabling constant-time detection of duplicate indices and thereby ensuring the time-side-channel security of the error vector generation process. Chen and Chou [10] pointed out that, as long as timing information related to the index values themselves is not leaked, sorting algorithms that may reveal ordering information through execution time are permissible. In other words, any comparison-based sorting algorithm is acceptable from a security perspective. They replaced the sorting network with a quicksort algorithm to perform index sorting, achieving improved performance in error vector generation. We reproduce this approach on our ARMv8 platform and observe that the performance gain is marginal, yielding an improvement of approximately 1 us for the McEliece6688128.

2) *Computation of $Te^{(1)}$* : In the Classic McEliece algorithm, the core operation of syndrome computation is the matrix-vector multiplication $Te^{(1)}$. In the `vec` implementation, this operation is performed by sequentially computing the inner product between each row of matrix T and the vector $e^{(1)}$. Specifically, for each row of T , a 64-bit accumulator is initialized to zero; each 64-bit word of the row is then bitwise ANDed with the corresponding word of $e^{(1)}$, and the results are accumulated using bitwise XOR operations. Finally, a series of shift and XOR operations are applied to compute the parity of the accumulator, which represents the inner product result of that row. However, this implementation suffers from considerable memory access overhead. During the computation of the inner product for the i -th row, the entire vector $e^{(1)}$ must be reloaded from memory. Consequently, $e^{(1)}$ is repeatedly loaded $(n-k)$ times throughout the matrix-vector

multiplication, significantly increasing the number of load instructions, imposing high memory pressure, and reducing overall computational efficiency.

To mitigate the overhead caused by redundant memory loads, we propose a batch-loading strategy. The key idea is that, instead of processing one matrix row at a time, multiple rows of T are processed simultaneously when a 64-bit word of $e^{(1)}$ is loaded. This approach reduces the number of vector loads by a factor of x , where x denotes the batch size. Specifically, x independent 64-bit accumulators are maintained in registers, each corresponding to one matrix row, enabling parallel computation of x row-wise inner products within a single iteration. Furthermore, unlike the `vec` implementation, which performs parity reduction independently for each row, we redesign the reduction procedure to perform parity reduction on x accumulators concurrently, thereby obtaining x inner product results in parallel. Figure 2 illustrates the schematic of the proposed batch-loading strategy for $x = 2$ and $x = 4$.

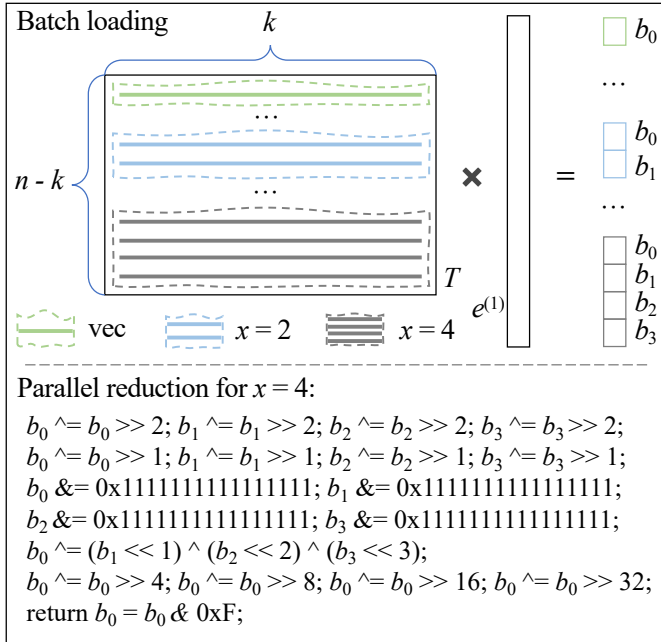


Fig. 2. Batch-loading strategy for $Te^{(1)}$ in syndrome.

For instance, in the case of two-way parallel reduction, the `vec` implementation requires $2 \times 6 = 12$ instructions, whereas the proposed two-way scheme requires only 11 instructions. Similarly, for four-way parallel reduction, the `vec` implementation requires $4 \times 6 = 24$ instructions, while the optimized design reduces this to 19 instructions, thereby significantly decreasing the total number of logical and shift operations. The average execution times for syndrome computation under different batch sizes x are presented in Table IV.

D. Decapsulation

The decapsulation process constitutes the core stage in which the receiver reconstructs the shared secret from the ciphertext. Built upon the classical McEliece construction over Goppa codes, this mechanism ensures post-quantum security

TABLE IV
EXECUTION TIME FOR SYNDROME UNDER DIFFERENT x

parameter set	vec (us)	2× parallel (us)	4× parallel (us)
McEliece348864	23.593	19.239	17.245
McEliece460896	79.361	40.496	36.418
McEliece6688128	89.17	79.199	74.24
McEliece6960119	131.849	85.188	84.167
McEliece8192128	105.258	92.688	94.267

while heavily relying on efficient error-correction decoding. Specifically, the receiver first applies the inverse of the secret permutation matrix P and the secret system matrix S to transform the received ciphertext c into a form c' that can be decoded by the secret Goppa code. Subsequently, by exploiting the algebraic structure of the Goppa polynomial $g(x)$ contained in the private key, the Berlekamp–Massey error-correction decoding algorithm is employed to compute and recover the original error vector e . Finally, the recovered error vector e is processed through a cryptographically secure hash function Hash, deterministically deriving the shared session key $K' = \text{Hash}(e)$. We present the optimization of the decapsulation process on the ARMv8 platform.

1) *Field Multiplication*: Arithmetic operations over the finite field $\mathbb{F}_{2^m}$ (where $m \in \{12, 13\}$), particularly finite-field multiplication, constitute one of the core computational modules in the decoding algorithm. Specifically, the field $\mathbb{F}_{2^{12}}$ is constructed as $\mathbb{F}_2[x]/(x^{12} + x^3 + 1)$, and $\mathbb{F}_{2^{13}}$ as $\mathbb{F}_2[x]/(x^{13} + x^4 + x^3 + x + 1)$. Under these representations, any field element can be regarded as an m -degree polynomial over $\mathbb{F}_2$. In the `vec` implementation, two representations of field elements are adopted: the bitsliced representation and the radix-16 representation. The radix-16 representation is used exclusively in the Berlekamp–Massey decoding algorithm, while the bitsliced representation is extensively employed in multiple stages, including Goppa polynomial evaluation, matrix elimination, and field arithmetic. Consequently, our optimization primarily targets finite-field multiplication under the bitsliced representation.

In the `vec` implementation, field multiplication $h(x) = f(x) \cdot g(x) \bmod p(x)$, where $f(x), g(x) \in \mathbb{F}_2[x]$ are two m -degree polynomials and $p(x)$ denotes the corresponding irreducible polynomial, is carried out via a double-loop convolution. During the convolution stage, for each pair $(i, j = 0, \dots, m - 1)$, the intermediate buffer is updated as $\text{buf}[i + j] \oplus = f[i] \& g[j]$, where bitwise AND and XOR operations respectively represent multiplication and addition over $\mathbb{F}_2$. The modular reduction is then performed sequentially from high to low degrees, folding overflow coefficients back according to the structure of $p(x)$, thereby yielding the final product in $\mathbb{F}_{2^m}$.

Intra-Multiplication Vector Parallelism (IMVP). To fully exploit the SIMD parallelism of the ARMv8 NEON instruction set, we vectorized the field multiplication routine by optimizing the data-level parallelism in the convolution phase, while preserving the original modular reduction logic to maintain mathematical equivalence. The key design principle is to transform the nested loop into a “broadcast-parallel AND-parallel XOR” pipeline, matching the NEON SIMD

execution model. The temporary buffer is first 16-byte aligned to enable aligned load and store operations without additional overhead. In the outer loop, the current multiplier $g[j]$ is fixed and broadcast across two 64-bit lanes of a 128-bit NEON register using the `vdupq_n_u64` instruction, thereby eliminating redundant loads within the inner loop. The inner loop loads two consecutive coefficients ($f[i], f[i+1]$) of $f(x)$ in parallel, performs dual-lane bitwise AND using `vandq_u64`, and accumulates the results into the intermediate buffer via `veorq_u64`. The updated results are then stored back using `vst1q_u64`, enabling pipelined updates over two 64-bit data blocks. For odd $m = 13$, the remaining coefficient is processed via a scalar fallback path to ensure complete coverage and numerical consistency.

This vectorized design effectively compresses the original $O(m^2)$ scalar logic operations into approximately $O(m^2/2)$ vectorized instructions, processing two 64-bit products per iteration and fully utilizing the throughput of 128-bit NEON registers. The alignment strategy and branch-free logic design eliminate pipeline stalls and memory access discontinuities, thereby enhancing instruction-level parallelism. All operations are implemented using constant-time logical instructions, independent of input data characteristics.

Inter-Multiplication Batch Parallelism (IMBP). In the decapsulation implementation, we observe that multiple instances exhibit a recurring computational pattern in which two field multiplications (`vec_mul`) are executed consecutively. Building upon the intra-multiplication vector parallelism described earlier, we further propose an inter-multiplication batch parallelism strategy, which performs two independent field multiplications within a single function (`vec_mul_2x`) call. This design achieves operation-level data parallelism and resource sharing by leveraging the 128-bit NEON SIMD architecture of ARMv8. Specifically, two input polynomial pairs, (f_1, g_1) and (f_2, g_2) , are mapped to the two 64-bit lanes of a single NEON register, thereby realizing a lane-level fusion layout, as illustrated in Fig. 3.

During the convolution phase, the algorithm employs a nested loop structure, where the outer loop fixes the current multiplier and the inner loop performs simultaneous partial accumulation for both multiplications using bitwise AND (`vandq_u64`) and XOR (`veorq_u64`) operations. Since NEON instructions can process both 64-bit lanes in parallel within a single clock cycle, this structure effectively reduces loop iteration counts and register scheduling overhead. Moreover, it allows the compiler to automatically unroll the loop to enhance pipeline parallelism. The modular reduction phase follows the same logic as in the `vec` implementation, folding higher-degree coefficients back into lower-degree positions through XOR-based feedback, while both SIMD lanes execute the reduction operations concurrently. Consequently, the results in lanes 0 and 1 correspond precisely to the independent outputs h_1 and h_2 , ensuring mathematical and structural equivalence with the `vec` baseline. The entire computation executes without synchronization or conditional branching, thereby maintaining strict constant-time behavior.

The proposed strategy achieves cross-operation parallelism, enabling concurrent execution of multiple independent mul-

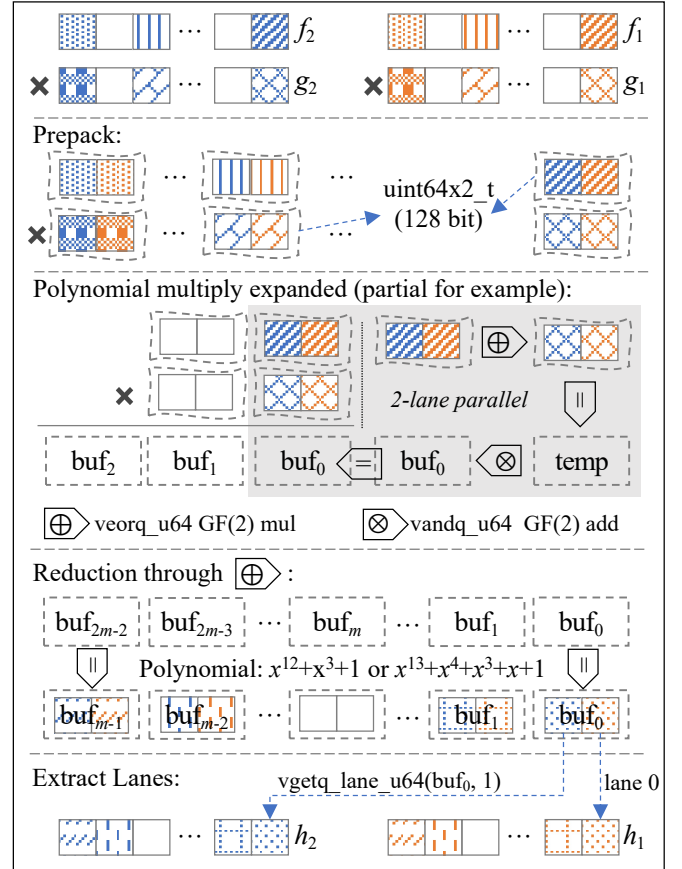


Fig. 3. Structure of the inter-multiplication batch parallelism (`vec_mul_2x`).

TABLE V
EXECUTION TIME FOR FIELD MULTIPLICATION

	single vec (ns)	IMVP (ns)	dual vec (ns)	IMBP (ns)
$\mathbb{F}_{2^{12}}$	151	145	300	178
$\mathbb{F}_{2^{13}}$	158	153	317	198

tifications at the instruction level. By merging two multiplication instances into a single NEON pipeline execution, the optimization significantly reduces function call overhead and memory accesses, improves cache locality, and enhances pipeline utilization. As a result, the proposed strategy delivers notable performance improvements in the decapsulation phase while preserving the correctness and security properties.

Table V presents the average execution time of field multiplications over $\mathbb{F}_{2^{12}}$ and $\mathbb{F}_{2^{13}}$ on the same ARMv8 platform. The comparison includes the baseline `vec` implementation, the single-operation optimization with IMVP, the dual independent `vec` execution (dual vec), and the proposed IMBP approach. Compared to dual independent execution, IMBP leverages cross-operation batch parallelism to enhance instruction throughput and data-channel utilization, resulting in nearly a 40% reduction in total execution time.

2) Dual-Lane Butterfly Fusion: In the Classic McEliece cryptosystem, the finite-field Fast Fourier Transform (FFT) serves as a key computational module in both private-key

polynomial evaluation and codeword mapping. The fundamental computational unit within the FFT is the butterfly operation, which follows a hierarchical structure and performs finite-field multiplications and XOR-based updates on node pairs (a_k, a_{k+s}) at each layer, with s denoting the stride. A typical FFT module in Classic McEliece includes both the additive FFT and the transposed additive FFT. The hierarchical computation structure, illustrated in Fig. 4 (using a four-layer example), proceeds left-to-right for the additive FFT and right-to-left for the transposed additive FFT, where each butterfly unit operates on polynomials over the field $\mathbb{F}_2[x]$. Although this serial execution model offers conceptual simplicity and implementation clarity, it suffers from frequent memory accesses, limited instruction-level parallelism, and suboptimal pipeline utilization, which together constrain the overall performance of the FFT module.

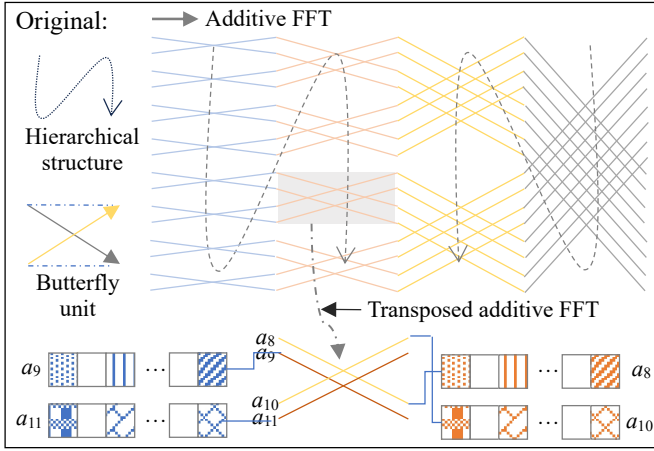
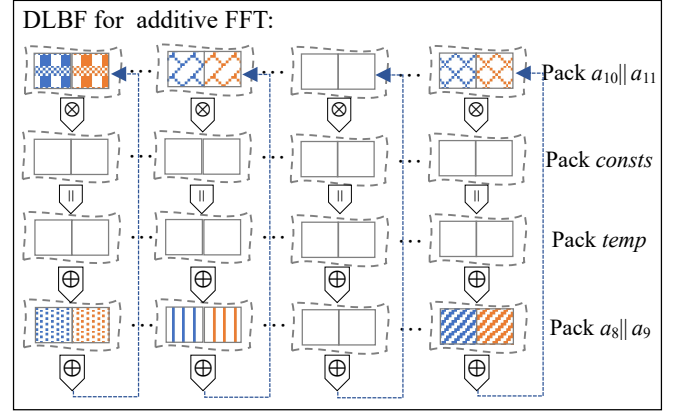


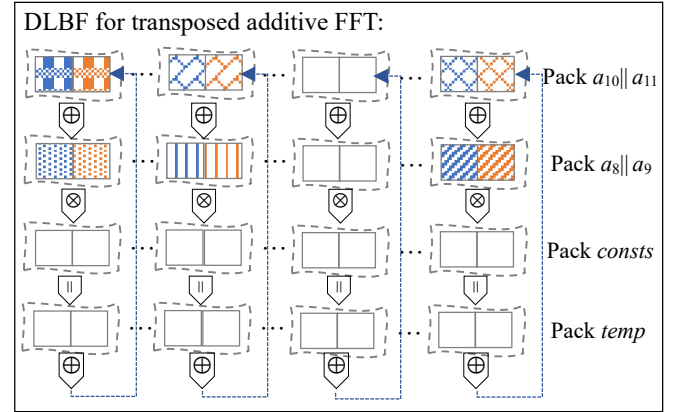
Fig. 4. Hierarchical structure of additive and transposed additive FFT.

To fully exploit the data-level parallelism available on the ARMv8 architecture, we introduce a Dual-Lane Butterfly Fusion (DLBF) optimization strategy. DLBF enhances both data parallelism and computational fusion by restructuring the iterative organization of butterfly layers, thereby reducing redundant memory access and function call overhead. The central idea of DLBF is to reorganize the computation pattern of butterfly layers such that adjacent butterfly units are fused and executed concurrently across two parallel data lanes, while strictly preserving algorithmic equivalence and numerical consistency. Figures 5 illustrate the dual-lane fusion process for both the additive FFT and the transposed additive FFT, respectively. Specifically, the inner loop of the FFT is reformulated so that each iteration simultaneously processes two adjacent input pairs, (a_k, a_{k+s}) and (a_{k+1}, a_{k+1+s}) . This dual-lane fusion enables field multiplications to be executed in a batched form, allowing both the multiplication and subsequent XOR operations for two butterfly pairs to be performed within the same instruction stream. Consequently, the instruction scheduling overhead and register access latency are significantly reduced.

Our DLBF replaces the original single-lane multiplication function (`vec_mul`) with a dual-lane parallel version (`vec_mul_2x`), enabling two independent multiplication op-



(a) DLBF execution mechanism for additive FFT



(b) DLBF execution mechanism for transposed additive FFT

Fig. 5. Dual-Lane Butterfly Fusion (DLBF) execution mechanism.

TABLE VI
EXECUTION TIME FOR BUTTERFLY OPERATION

	butterfly (us)	ours (us)	butterfly_tr (us)	ours_tr (us)
$\mathbb{F}_{2^{12}}$	23.612	16.983	23.626	17.353
$\mathbb{F}_{2^{13}}$	52.844	35.953	61.248	44.725

erations to be executed within a single function call. This reduces both the number of function invocations and memory access instructions. The dual-lane execution mode not only introduces spatial parallelism in computation but also enhances pipeline utilization efficiency. By fusing adjacent computation paths, DLBF establishes a butterfly-aligned execution pattern that tightens inter-layer data dependencies and minimizes intermediate memory load/store operations. Theoretically, the conventional FFT butterfly layer exhibits a time complexity of $O(\ell \cdot n)$, where ℓ denotes the number of layers and n the input vector length. Through dual-lane parallelism, DLBF effectively halves the number of active computational units per layer, thereby reducing the overall computational complexity to approximately $O(\ell \cdot n/2)$.

As shown in Table VI, the proposed DLBF approach yields substantial performance gains over the official `vec` implementation on the same ARMv8 platform. Maintaining full numerical equivalence and algorithmic correctness, DLBF achieves

superior efficiency by combining structural reorganization with computation fusion. This optimization effectively overcomes the traditional bottlenecks of the FFT butterfly layer in terms of parallelism and memory performance.

IV. PERFORMANCE EVALUATION

This section presents an analysis of the experimental results and compares them with related implementations.

A. Experimental Setup

To systematically evaluate the performance and portability of the proposed optimization schemes, comprehensive experiments were conducted on three representative ARMv8-based platforms: the Kunpeng 920 processor, and two MacBook Pros equipped with Apple M1 and Apple M2 Pro chips, respectively. These platforms differ significantly in terms of microarchitectural design, core configuration, cache hierarchy, and memory organization.

The Kunpeng 920 processor operates at a maximum frequency of 2.6 GHz. It features a three-level cache hierarchy (L1: 64 KB, L2: 512 KB, L3: 32 MB). The experiments were performed under CentOS Linux release 8.5.2111, using the GCC 8.5 compiler with optimization flags `-O3 -march=armv8-a+crypto`. The Apple M1 processor operates at frequencies up to 3.2 GHz. It provides a 128-bit wide NEON SIMD execution unit and employs a unified memory architecture. The experimental environment was based on macOS 26.0.1, compiled with Clang 17.0.0 using the flags `-O3 -arch arm64`. The Apple M2 Pro processor adopts the second-generation ARMv8.5 architecture with a peak frequency of 3.5 GHz. It retains a 128-bit SIMD width while improving memory bandwidth to 200 GB/s. Experiments were conducted on macOS 15.0.1 using the Clang 16.0.0 compiler with the same optimization flags `-O3 -arch arm64`.

Performance evaluation focused on two primary metrics:

- **Execution Time:** measuring the average latency of key generation, encapsulation, and decapsulation to assess overall computational efficiency;
- **Speedup:** quantifying the relative performance gain of the optimized implementation.

All experiments were executed in single-threaded mode to eliminate interference from multicore scheduling. Each configuration was executed 1,000 iterations to ensure statistically stable average measurements.

B. Performance Analysis

Table VII summarizes the performance of the Classic McEliece on the Kunpeng 920, Apple M1, and Apple M2 Pro platforms across multiple parameter sets. For each platform and parameter configuration, the table reports the average execution time of key generation, encapsulation, and decapsulation, along with the corresponding speedups over the official `vec` implementation. Overall, the proposed optimizations yield performance improvements on all three platforms. However, the magnitude of acceleration varies notably with both platform characteristics and parameter sizes.

For key generation (Keygen), significant speedups are observed for most parameter sets, with the largest improvements achieved on the Apple platforms for large-scale configurations (e.g., McEliece8192128: $1.90\times$ on M1 and $2.04\times$ on M2 Pro). The Kunpeng 920 also demonstrates considerable acceleration ($1.33\times$). KeyGen involves extensive operations on dense and sparse binary matrices, including row manipulations, column permutations, and matrix-matrix multiplications, and is therefore highly sensitive to memory bandwidth, cache hierarchy, and locality in memory accesses. It additionally contains substantial bitwise logic (XOR/AND) and serial row dependencies. For extremely large parameters, the high-bandwidth unified memory and superior single-core SIMD throughput of the Apple processors more effectively leverage our vectorized elimination routines, masked row-swapping mechanism, and batched application of (P, L^{-1}, U^{-1}) . In contrast, the acceleration on Kunpeng is constrained by the limitations of its single-threaded memory subsystem, particularly in handling high-frequency bitwise updates and non-contiguous write operations, which expose bottlenecks in vector-level concurrency and writeback latency.

For encapsulation (Encaps.), the speedup for encapsulation is generally smaller than that of KeyGen and decapsulation. However, on the Kunpeng 920, it consistently exceeds that on the Apple platforms (e.g., for McEliece348864: $1.34\times$ on Kunpeng versus $1.14\times$ on M1 and $1.09\times$ on M2 Pro). One major bottleneck in encapsulation is the matrix-vector multiplication $Te^{(1)}$. In the official `vec` implementation, the vector $e^{(1)}$ is reloaded nearly $(n - k)$ times across the computation, leading to inefficient cache reuse. Our batch-loading mechanism significantly reduces redundant memory accesses. On Kunpeng, the relatively large L3 cache and suboptimal cache reuse in the reference implementation make this reduction especially beneficial, effectively decreasing L3-memory traffic and resulting in stronger acceleration. Conversely, the unified on-chip memory system and higher memory bandwidth of the Apple platforms already provide substantial vector-loading throughput, thereby limiting the relative gains from reducing redundant loads.

For decapsulation (Decaps.), across most parameter sets, decapsulation exhibits moderate to substantial improvements (e.g., McEliece460896 achieves $1.62\text{--}1.69\times$ on the Apple platforms). Speedups on the Apple M1/M2 Pro are consistently higher than those on the Kunpeng 920. Decapsulation involves a large number of finite-field multiplications and FFT-related routines—both computation-intensive kernels with considerable intrinsic instruction-level parallelism that is underexploited in the reference implementation due to limited vectorization and register utilization. The Apple SoCs provide stronger support for high-throughput SIMD execution and benefit from a unified memory architecture; thus, when computation-fusion and dual-lane parallelism are applied, the Apple processors more effectively translate these structural optimizations into higher instruction throughput. Furthermore, the more advanced register renaming and out-of-order execution capabilities of the Apple cores help avoid register spilling in the dual-path batch multiplication routines, resulting in superior performance gains compared with Kunpeng.

TABLE VII
EXECUTION TIME AND SPEEDUP OF CLASSIC McELIECE KEM ON THREE ARMv8 PLATFORMS

parameter set	operation	Kunpeng 920			Apple M1			Apple M2 Pro		
		vec (us)	ours (us)	speedup	vec (us)	ours (us)	speedup	vec (us)	ours (us)	speedup
McEliece348864	Keygen	101,893.70	73,966.65	1.38 ×	38,032.22	30,866.56	1.23×	36,876.86	30,002.39	1.23×
	Encaps.	41.29	30.86	1.34 ×	17.81	15.62	1.14×	16.79	15.43	1.09×
	Decaps.	225.06	186.09	1.21×	100.38	71.22	1.41 ×	94.91	72.80	1.30×
McEliece460896	Keygen	335,749.55	274,566.18	1.22×	125,652.40	97,371.22	1.29×	123,401.32	92,585.36	1.33×
	Encaps.	89.70	69.19	1.30×	37.96	34.93	1.09×	37.65	34.52	1.09×
	Decaps.	572.14	459.61	1.24×	271.59	161.07	1.69×	263.48	162.69	1.62×
McEliece6688128	Keygen	833,737.01	651,423.80	1.28×	201,273.80	177,083.41	1.14×	200,284.60	174,993.55	1.14×
	Encaps.	152.69	122.59	1.25×	63.44	59.86	1.06×	60.98	58.73	1.04×
	Decaps.	631.66	489.00	1.29 ×	296.80	175.68	1.69×	288.48	171.82	1.68×
McEliece6960119	Keygen	414,141.62	376,488.12	1.10×	161,617.15	130,858.52	1.24×	163,361.47	132,982.66	1.23×
	Encaps.	153.84	126.18	1.22×	51.88	50.32	1.03×	52.55	50.84	1.03×
	Decaps.	615.08	489.94	1.26×	299.00	172.64	1.73×	284.41	169.40	1.68 ×
McEliece8192128	Keygen	799,830.25	600,361.60	1.33×	288,037.27	151,840.50	1.90 ×	288,948.90	141,487.72	2.04 ×
	Encaps.	152.69	114.95	1.33×	51.42	41.68	1.23 ×	49.75	39.34	1.26 ×
	Decaps.	630.76	497.89	1.27×	298.57	177.69	1.68 ×	293.97	176.83	1.66×

TABLE VIII
PERFORMANCE COMPARISON BETWEEN LIBOQS AND THE OPTIMIZED IMPLEMENTATION OF CLASSIC McELIECE KEM

parameter set	operation	Kunpeng 920			Apple M1			Apple M2 Pro		
		liboqs (us)	ours (us)	speedup	liboqs (us)	ours (us)	speedup	liboqs (us)	ours (us)	speedup
McEliece348864	Keygen	157,806.40	73,966.66	2.13×	233,775.71	30,866.56	7.57×	65,796.23	30,002.39	2.19×
	Encaps.	78.17	30.86	2.53×	17.59	15.62	1.13×	17.23	15.43	1.12×
	Decaps.	28,400.80	186.09	152.62×	18,194.49	71.22	255.47×	16,895.11	72.80	232.06×
McEliece460896	Keygen	666,941.40	274,566.18	2.43×	780,842.00	97,371.23	8.02×	168,601.61	92,585.36	1.82×
	Encaps.	162.49	69.20	2.35×	39.42	34.93	1.13×	36.31	34.53	1.05×
	Decaps.	56,862.09	459.61	123.72×	41,571.82	161.08	258.09×	33,235.01	162.69	204.28×
McEliece6688128	Keygen	693,785.80	651,423.80	1.07×	1,428,639.50	177,083.41	8.07×	709,296.40	174,993.56	4.05×
	Encaps.	289.44	122.59	2.36×	89.29	59.87	1.49×	89.13	58.73	1.52
	Decaps.	108,744.39	489.01	222.38×	79,789.37	175.69	454.16×	64,436.30	171.83	375.00
McEliece6960119	Keygen	786,785.00	376,488.13	2.09×	800,653.25	130,858.52	6.12×	547,515.67	132,982.67	4.12×
	Encaps.	964.77	126.18	7.65×	116.98	50.32	2.32×	609.09	50.85	11.98×
	Decaps.	105,243.38	489.94	214.81×	77,192.33	172.64	447.12×	61,494.98	169.41	363.00×
McEliece8192128	Keygen	878,797.50	600,361.60	1.46×	1,532,301.00	151,840.50	10.09×	597,787.17	141,487.73	4.23×
	Encaps.	334.70	114.96	2.91×	79.82	41.69	1.91×	68.40	39.34	1.74×
	Decaps.	132,798.09	497.89	266.72×	97,574.19	177.70	549.10×	77,233.00	176.83	436.76×

C. Comparison with Liboqs Implementations

Liboqs is a widely recognized open-source library that provides production-grade implementations of PQC algorithms. Its credibility stems from strict adherence to NIST guidelines and extensive validation across both academic and industrial environments. In addition to supporting multiple classes of post-quantum primitives, liboqs features a modular design and broad cross-platform compatibility (e.g., x86 and ARM architectures), thereby facilitating efficient integration and optimization. Table VIII presents a comprehensive performance comparison between the liboqs implementation and the optimized version developed in this work, demonstrating the applicability and performance benefits of the proposed techniques across various platforms and parameter sets.

Experimental results show that the proposed implementation consistently outperforms liboqs on all evaluated platforms and operations, with performance improvements exhibiting a stable trend across architectures. On the Kunpeng 920 platform, the optimized implementation achieves a 1.07×–2.43× speedup for key generation, a 2.35×–7.65× speedup for encapsulation, and the most substantial improvement for decapsulation, reaching 123.72×–266.72×. On the Apple M1 platform, the

performance gains are even more pronounced: key generation attains a 6.12×–10.09× speedup, encapsulation achieves 1.13×–2.32×, and decapsulation reaches 255.47×–549.10×. On the Apple M2 Pro, the speedups for key generation, encapsulation, and decapsulation range from 1.82×–4.23×, 1.05×–11.98×, and 204.28×–436.76×, respectively. Collectively, these results clearly demonstrate that the proposed optimization framework provides substantial efficiency improvements for Classic McEliece in realistic deployment environments when compared with liboqs.

It is worth noting that, although liboqs incorporates hardware acceleration mechanisms on multiple platforms (e.g., AVX2 or NEON SIMD), its ARM implementation of Classic McEliece remains primarily based on the `ref` version and does not include a dedicated ARMv8/NEON-optimized code path. In contrast, our implementation performs systematic, architecture-aware vectorization and cross-module fusion optimizations tailored specifically to the ARMv8 platform, thereby achieving significantly higher performance than liboqs on ARMv8-based systems.

In addition, the PQCclean [20] library also includes an implementation of Classic McEliece. However, our examination

TABLE IX
PERFORMANCE COMPARISON WITH RELATED WORK

work	platform	scheme	Key Generation (us)	Encapsulation (us)	Decapsulation (us)
Kwon et al. [21]	Apple A10X @ 2.38 GHz	FrodoKEM640	1,035,600	1,108,900	1,117,700
Filho et al. [22]	Apple M1 @ 3.2 GHz	FrodoKEM640	157.50	199.69	194.69
		FrodoKEM976	306.88	358.75	339.72
		FrodoKEM1344	519.38	598.75	574.69
Nguyen et al. [23]	Apple M1 @ 3.2 GHz	Lightsaber	9.75	11.63	11.03
		Saber	16.03	18.72	18.13
		Firesaber	24.06	27.47	27.09
		Kyber512	7.19	10.19	9.19
		Kyber768	11.34	15.38	14.28
		Kyber1024	17.47	22.38	20.97
Chen and Chou [10]	stm32f4-Discovery @ 168MHz	McEliece348864	12,779,357.34	3,465.47	16,111.20
		McEliece460896	-	6,436.52	38,899.92
Official (AVX) [24]	Intel Haswell @ 3.10GHz	McEliece348864	18,292.22	12.12	41.18
		McEliece460896	49,440.71	26.23	85.04
		McEliece6688128	143,144.19	56.71	99.01
		McEliece6960119	102,256.60	47.48	92.65
		McEliece8192128	156,837.19	50.99	100.25
Ours	Apple M1 @ 3.2 GHz	McEliece348864	30,866.56	15.62	71.22
		McEliece460896	97,371.23	34.93	161.08
		McEliece6688128	177,083.41	59.87	175.69
		McEliece6960119	130,858.52	50.32	172.64
		McEliece8192128	151,840.50	41.69	177.70

shows that its implementation closely follows the four official implementations and does not incorporate any ARMv8-specific optimizations. As a result, its fastest implementation achieves performance comparable to the official `vec` version, whereas the optimized implementation presented in our work consistently delivers substantially superior performance relative to PQClean.

D. Comparison with Related Work

Existing studies of the Classic McEliece on ARMv8 platforms remain relatively limited. Sim et al. reported in a preprint what they described as “the first implementation of Classic McEliece on ARMv8.” However, their optimization efforts focus solely on the encapsulation and decapsulation, and the experimental evaluation provides only partial performance results for the encapsulation pipeline (up to the ENCODE) on the Apple M1 processor. In subsequent work, the same group introduced NEON-based vector parallelization for finite-field operations in Classic McEliece, employing 64-bit ARM vector registers to construct field multipliers for the first time. Nevertheless, these studies do not deliver a systematic optimization of the full Classic McEliece workflow, nor do they report comprehensive performance measurements for complete encapsulation or decapsulation procedures. As a result, the overall effectiveness and generality of their optimization strategies cannot be fully assessed.

Table IX presents a performance comparison of Classic McEliece and other PQC KEM schemes on the Apple M1 platform across different hardware configurations. The optimized implementation by Chen and Chou for ARM platforms exhibits relatively lower performance, primarily due to the lack of advanced ARMv8 features in ARMv7, such as wider SIMD vector support and more efficient memory access, which results in bottlenecks during large-scale matrix operations. In

contrast, the ARMv8 platform leverages its enhanced computational and memory optimization capabilities, leading to significant performance improvements.

Regarding the key generation operation, the Classic McEliece KEM typically experiences longer key generation times, especially for larger parameter sets, with key generation times being notably higher than those of lattice-based KEM schemes. For example, Nguyen et al. [23] demonstrated that, on the Apple M1 platform, the key generation time for Kyber512 is only 7.19 us, and for Kyber1024 it is approximately 17.47 us. In comparison, the key generation time for Classic McEliece with the McEliece348864 parameter set is 30,866.56 us, showing a significant difference. This discrepancy primarily reflects the inherent computational complexity disadvantage of the error-correcting code-based mathematical structure of Classic McEliece. Nevertheless, it is noteworthy that despite the longer key generation time, the optimized version presented in this work on the Apple M1 platform approaches the performance of the official AVX2 vectorized implementation on the Intel Haswell platform, indicating that, with vectorization optimization, Classic McEliece still holds significant potential for performance improvement on ARM architectures.

In the encapsulation operation, Classic McEliece exhibits a performance gap compared to lattice-based schemes such as Kyber and Saber. Lattice-based schemes typically maintain high efficiency during encapsulation due to their ability to exploit structural properties for more efficient computation. However, Classic McEliece on the Apple M1 platform demonstrates a time of approximately 15.62 us for encapsulation, which is significantly better than FrodoKEM-640’s encapsulation time of 199.69 us, illustrating a notable performance advantage, especially for larger parameter sizes. During the decapsulation phase, although Classic McEliece

still lags behind lattice-based schemes like Kyber and Saber, it shows considerable potential in comparison to FrodoKEM. For instance, FrodoKEM-640's encapsulation and decapsulation times are 199.69 us and 194.69 us, respectively, while the decapsulation time for McEliece348864 is only about one-third of FrodoKEM's. Therefore, Classic McEliece demonstrates competitive performance in the decapsulation phase and reveals significant potential for future deployment.

Overall, these results provide a consistent view of the performance landscape of Classic McEliece on modern ARMv8 processors. The comparison across different KEMs and hardware platforms highlights both the algorithm's inherent computational characteristics and the substantial gains enabled by architecture-aware vectorization. Despite the relatively high cost of key generation, the optimized implementation demonstrates competitive encapsulation and decapsulation performance, narrowing the gap with lattice-based schemes while outperforming unstructured-lattice KEMs. These findings confirm that significant performance improvements can be achieved on ARMv8 platforms without modifying the underlying algorithmic structure, underscoring the practicality and scalability of the proposed optimization strategy.

V. CONCLUSION

In this work, we presented a comprehensive, architecture-aware optimization framework (RACE) targeting the primary performance bottlenecks of the Classic McEliece on ARMv8 platforms. For the key generation phase, we introduced an LUP-decomposition-based implicit data-layout optimization and applied NEON-SIMD vectorization to the core routines, substantially improving matrix-operation efficiency. In the encapsulation phase, we optimized the error-vector generation process and developed a batch-loading strategy to reduce memory redundancy in the matrix-vector multiplication, thereby enhancing overall throughput. Furthermore, in the decapsulation phase, field multiplication and butterfly operations were parallelized across NEON lanes, accelerating the execution of these computation-intensive kernels. Collectively, these optimizations alleviate the algorithmic and architectural bottlenecks of Classic McEliece and deliver significant speed improvements on ARMv8 processors.

REFERENCES

- [1] M. Suárez-Albela, T. M. Fernández-Caramés, P. Fraga-Lamas, and L. Castedo, "A practical performance comparison of ecc and rsa for resource-constrained iot devices," in *2018 Global Internet of Things Summit (GloTS)*, pp. 1–6, 2018.
- [2] D. J. Bernstein, T. Chou, T. Lange, I. von Maurich, R. Misoczki, R. Niederhagen, E. Persichetti, C. Peters, P. Schwabe, N. Sendrier, et al., "Classic mceliece: conservative code-based cryptography," *NIST submissions*, vol. 1, no. 1, pp. 1–25, 2017.
- [3] G. Alagic, M. Bros, P. Ciadoux, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, et al., *Status report on the fourth round of the nist post-quantum cryptography standardization process*. US Department of Commerce, National Institute of Standards and Technology, 2025.
- [4] P. Sanal, E. Karagoz, H. Seo, R. Azarderakhsh, and M. Mozaffari-Kermani, "Kyber on arm64: Compact implementations of kyber on 64-bit arm cortex-a processors," in *Security and Privacy in Communication Networks* (J. Garcia-Alfaro, S. Li, R. Poovendran, H. Debar, and M. Yung, eds.), (Cham), pp. 424–440, Springer International Publishing, 2021.
- [5] J. Ortiz, F. C. Rodrigues, D. G. Filho, C. Teixeira, J. López, and R. Dahab, "Evaluation of crystals-kyber and saber on the armv8 architecture," in *Anais do XXII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais*, (Porto Alegre, RS, Brasil), pp. 372–377, SBC, 2022.
- [6] L. Zhao, J. Zhang, J. Huang, Z. Liu, and G. Hancke, "Efficient implementation of kyber on mobile devices," in *2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS)*, pp. 506–513, 2021.
- [7] A. C. Aguilera, I. T. Monroy, J. J. Vegas Olmos, and J. L. Imaña, "Arm architecture optimizations for line-rate pqc communications," in *2025 International Conference on Optical Network Design and Modeling (ONDM)*, pp. 1–4, 2025.
- [8] Y. Kim, J. Song, and S. C. Seo, "Accelerating falcon on armv8," *IEEE Access*, vol. 10, pp. 44446–44460, 2022.
- [9] H. Kwon, K. Jang, H. Kim, H. Kim, M. Sim, S. Eum, W.-K. Lee, and H. Seo, "Armed frodo," in *Information Security Applications* (H. Kim, ed.), (Cham), pp. 206–217, Springer International Publishing, 2021.
- [10] M.-S. Chen and T. Chou, "Classic mceliece on the arm cortex-m4," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, p. 125–148, Jul. 2021.
- [11] J. Roth, E. Karatsiolis, and J. Krämer, "Classic mceliece implementation with low memory footprint," in *Smart Card Research and Advanced Applications* (P.-Y. Liardet and N. Mentens, eds.), (Cham), pp. 34–49, Springer International Publishing, 2021.
- [12] M. Sim, H. Kwon, S. Eum, G. Song, M. Lee, and H. Seo, "Efficient implementation of the classic mceliece on armv8 processors," in *Information Security Applications* (H. Kim and J. Youn, eds.), (Singapore), pp. 324–337, Springer Nature Singapore, 2024.
- [13] P.-J. Chen, T. Chou, S. Deshpande, N. Lahr, R. Niederhagen, J. Szefer, and W. Wang, "Complete and improved fpga implementation of classic mceliece," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, p. 71–113, Jun. 2022.
- [14] S. Chen, H. Lin, W. Huang, and Y. Huang, "Hardware design and implementation of classic mceliece post-quantum cryptosystem based on fpga," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–6, 2022.
- [15] H. Zhang, X. Qiao, J. Tian, S. Song, and Z. Wang, "Fast hardware architecture with efficient matrix computations for the key generation of classic mceliece," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 3, pp. 1321–1331, 2025.
- [16] C. Nugier and V. Migliore, "Acceleration of a classic mceliece postquantum cryptosystem with cache processing," *IEEE Micro*, vol. 44, no. 1, pp. 59–68, 2024.
- [17] D. Xu, Y. Gao, Y. Zhou, and J. Weng, "High-performance implementations of classic mceliece kem on gpus," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2025.
- [18] W. Wu, J. Dong, Z. Xu, Z. Dong, D. Duong, F. Xiao, and J. Lin, "Symphony of speeds: Harmonizing classic mceliece cryptography with gpu innovation," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 8746–8759, 2025.
- [19] K. Wang, "Arm v8 architecture and programming," in *Embedded and Real-Time Operating Systems*, pp. 505–792, Springer, 2023.
- [20] M. J. Kannwischer, P. Schwabe, D. Stebila, and T. Wiggers, "Improving software quality in cryptography standardization projects," in *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, (Los Alamitos, CA, USA), pp. 19–30, IEEE Computer Society, 2022.
- [21] H. Kwon, K. Jang, H. Kim, H. Kim, M. Sim, S. Eum, W.-K. Lee, and H. Seo, "Armed frodo: Frodokem on 64-bit armv8 processors," in *International Conference on Information Security Applications*, pp. 206–217, Springer, 2021.
- [22] D. L. G. Filho, G. Brandão, G. Adj, A. Alblooshi, I. A. Canales-Martínez, J. Chávez-Saab, and J. López, "Pqc-amx: Accelerating saber and frodokem on the apple m1 and m3 socs," in *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*, pp. 9–16, 2024.
- [23] D. T. Nguyen and K. Gaj, "Optimized software implementations of crystals-kyber, ntru, and saber using neon-based special instructions of armv8," in *Proceedings of the NIST 3rd PQC Standardization Conference (NIST PQC 2021)*, 2021.
- [24] C. McEliece, "Cycle Counts for the October 2022 software on an Intel Haswell CPU core." <https://classic.mceliece.org/impl.html>, 2022.